HUFE 数据库第二讲

2026 年 3 月 31 日

1 本次课定位

本次课对应 5 次课安排中的第 5 次课，时长 3 小时，内容覆盖**数据库安全性、数据库完整性、关系数据理论、数据库设计以及数据库恢复与并发控制**。这是数据库部分的收尾讲，也是整个课程的综合串讲。

这一讲的特点是：**概念分散但考点集中**。不同于第一讲可以用”系统—模型—操作”串起来，本讲更像五个相对独立的模块，需要分别抓住各模块的核心考点，再在最后串联成整体认知。

1.1 本次课目标

- 理解数据库安全性的基本概念，掌握自主存取控制中 GRANT 和 REVOKE 的基本用法。
- 掌握实体完整性、参照完整性和用户定义完整性的定义与约束规则。
- 理解函数依赖的定义，掌握 1NF、2NF、3NF 和 BCNF 的判断方法。
- 掌握数据库设计的六个基本步骤，能绘制简单 E-R 图并将其转换为关系模式。
- 理解事务的 ACID 属性，了解故障类型与恢复技术的基本思路，掌握并发控制的基本概念。

1.2 考试导向提醒

本讲在试卷中属于**分散高频区**，考法灵活，常见方式包括：

- **概念判断题**：如 ACID 属性、事务定义、安全性控制手段、完整性约束类型。
- **范式判断题**：给出关系模式和函数依赖集，判断属于第几范式并说明原因。
- **模式分解题**：将低范式关系模式分解成满足更高范式的模式组合。
- **E-R 图与关系模式转换题**：根据描述设计 E-R 图，再转换成关系模式。

这一讲的学习策略是：**每个模块各抓一条主线，不要混在一起背**。

© 2026 isomoes

1.3 3 小时建议节奏

阶段	时间	主要内容
第 1 段	30 分钟	数据库安全性：基本概念、自主存取控制、GRANT/REVOKE
第 2 段	30 分钟	数据库完整性：实体完整性、参照完整性、用户定义完整性
第 3 段	45 分钟	关系数据理论：函数依赖、范式（1NF～BCNF）、模式分解
第 4 段	30 分钟	数据库设计：六阶段、E-R 模型、关系模式转换
第 5 段	25 分钟	恢复与并发控制：事务 ACID、故障类型、恢复技术、并发问题

2 数据库安全性

2.1 为什么要有安全性控制

数据库中往往存储着大量敏感数据。安全性控制的目标是：**确保只有合法的用户以合法的方式访问数据**。

数据库安全性涉及多个层面，但考试主要关注 DBMS 层面的安全机制。

2.2 自主存取控制

最常见的数据库安全控制机制是**自主存取控制（DAC）**，其核心是”谁拥有数据，谁决定谁能访问”。

在 SQL 中通过两条语句实现：

- `GRANT`：授予权限；
- `REVOKE`：回收权限。

2.2.1 GRANT 授权

```sql
GRANT SELECT, INSERT
ON Student
TO user1
WITH GRANT OPTION;
```

关键点：

- `ON` 后跟被授权对象，如表名；
- `TO` 后跟被授权用户；
- `WITH GRANT OPTION` 表示允许 user1 再把此权限转授他人。

2.2.2 REVOKE 回收

```sql
REVOKE SELECT
ON Student
FROM user1
CASCADE;
```

`CASCADE` 表示同时回收 user1 转授给他人的相应权限。

2.3 强制存取控制

强制存取控制（MAC）基于安全标签，由系统强制执行，用户不能自行修改。常见于对安全要求极高的系统。考试中只需了解其概念，区分 DAC 和 MAC 即可。

2.4 数据库安全性练习

1. 自主存取控制和强制存取控制的主要区别是什么?
2. `GRANT SELECT ON Student TO user1;` 语句的作用是什么?
3. `WITH GRANT OPTION` 表示什么含义?
4. 若要回收 user1 对 Student 表的查询权限，应如何写 SQL 语句?
5. 请判断：自主存取控制中，数据对象的访问权限由 DBMS 强制决定，用户不能干预。（　　）
6. 请判断：使用 `CASCADE` 选项回收权限时，仅回收目标用户自己的权限，不影响其转授给他人的权限。（　　）

3 数据库完整性

3.1 完整性的意义

数据库完整性是指数据库中数据的**正确性、有效性和相容性**。它通过约束规则来防止不合语义的数据进入数据库。

完整性约束分为三类，这三类必须分清：

1. **实体完整性**
2. **参照完整性**
3. **用户定义完整性**

3.2 实体完整性

实体完整性规则：关系的主键属性值不能为空（NULL）。

这是因为主键的作用是唯一标识关系中的每一个元组，如果主键为空，就无法进行唯一标识。

在 SQL 中，定义主键时会自动施加此约束：

```sql
CREATE TABLE Student (
    Sno CHAR(9) PRIMARY KEY,
    Sname VARCHAR(20) NOT NULL,
    Sdept VARCHAR(20)
);
```

3.3 参照完整性

参照完整性规则：如果关系 R 的外键 F 是关系 S 的主键，则 R 中每个元组在 F 上的取值必须：

- 等于 S 中某个元组的主键值；
- 或为空值（如果允许外键取空值）。

例如选课表 $SC(Sno, Cno, Grade)$ 中，Sno 是外键，引用 $Student$ 表的主键。若 $Student$ 表中不存在某个 Sno，则 SC 中不能插入含该 Sno 的记录。

在 SQL 中定义外键约束：

```sql
CREATE TABLE SC (
    Sno CHAR(9),
    Cno CHAR(4),
    Grade SMALLINT,
    PRIMARY KEY (Sno, Cno),
    FOREIGN KEY (Sno) REFERENCES Student(Sno),
    FOREIGN KEY (Cno) REFERENCES Course(Cno)
);
```

3.4 用户定义完整性

用户定义完整性是针对具体应用语义的约束，例如：

- 成绩必须在 0 到 100 之间；
- 性别只能是” 男” 或” 女”。

在 SQL 中常用 `CHECK` 子句实现：

```sql
CREATE TABLE SC (
    Sno CHAR(9),
    Grade SMALLINT CHECK (Grade >= 0 AND Grade <= 100)
);
```

© 2026 isomoes

3.5 完整性约束命名

可以使用 CONSTRAINT 给约束命名，便于后续修改或删除：

```sql
CREATE TABLE SC (
    Grade SMALLINT,
    CONSTRAINT GradeCheck CHECK (Grade >= 0 AND Grade <= 100)
);
```

3.6 数据库完整性练习

1. 什么是实体完整性？为什么主键不能为空？
2. 参照完整性规则约束的是哪一类属性？
3. 外键的取值有哪些合法情况？
4. 用户定义完整性和前两类完整性的主要区别是什么？
5. CHECK 子句在 SQL 中的作用是什么？请举一个例子。
6. 请判断：外键的取值只能是被参照关系中已有的主键值，不允许为 NULL。（　　）

4 关系数据理论

4.1 为什么要有规范化理论

设计关系数据库时，如果关系模式设计不合理，就会出现：

- **数据冗余**：同样的数据被反复存储；
- **更新异常**：修改一处可能要修改多处；
- **插入异常**：无法单独插入某些数据；
- **删除异常**：删除某数据时意外丢失其他信息。

规范化理论的目标就是通过合理分解关系模式，消除这些问题。

4.2 函数依赖

4.2.1 定义

设关系模式 $R(U), X, Y \subseteq U$。若对 R 的任意两个元组 t_1, t_2，只要 $t_1[X] = t_2[X]$，就有 $t_1[Y] = t_2[Y]$，则称 X 函数确定 Y，记作 $X \to Y$，即 Y **函数依赖**于 X。

通俗理解：X **的值确定之后，**Y **的值也就唯一确定了。**

例如：

- 学号 $\to$ 学生姓名（已知学号，姓名唯一确定）；
- 学号 $\to$ 所在院系。

4.2.2 完全函数依赖与部分函数依赖

- **完全函数依赖**：$X \to Y$，且 X 的任何真子集都不能函数确定 Y。记作 $X \xrightarrow{F} Y$。
- **部分函数依赖**：$X \to Y$，但存在 X 的某个真子集 X' 使得 $X' \to Y$。记作 $X \xrightarrow{P} Y$。

例如：选课关系 $SC(Sno, Cno, Grade)$，$(Sno, Cno) \to Grade$ 是完全函数依赖，因为仅用学号或仅用课程号都不能唯一确定成绩。

4.2.3 传递函数依赖

若 $X \to Y$，$Y \to Z$，且 $Y \not\to X$，则称 Z 传递函数依赖于 X。
例如：如果学号确定系名，系名确定系主任，则系主任传递依赖于学号。

4.3 第一范式（1NF）

1NF：关系模式 R 的每个属性都是不可再分的原子值。
这是最基本的要求，所有关系数据库中的关系都必须满足 1NF。
判断要点：如果某一属性可以继续拆分成多个子属性，那么违反 1NF。

4.4 第二范式（2NF）

2NF：在满足 1NF 的基础上，每个非主属性**完全函数依赖**于主键（候选键）。
换句话说，若存在非主属性对主键的**部分函数依赖**，则不满足 2NF。
例如：若关系模式为 $R(Sno, Cno, Sname, Grade)$，其中 (Sno, Cno) 是主键，但 $Sno \to Sname$（学号已能确定姓名），$Sname$ 部分依赖于主键，则 R 不满足 2NF。
解决方法：分解为：

- $Student(Sno, Sname)$；
- $SC(Sno, Cno, Grade)$。

4.5 第三范式（3NF）

3NF：在满足 2NF 的基础上，每个非主属性**不传递依赖**于主键。
若存在非主属性对主键的**传递函数依赖**，则不满足 3NF。
例如：若 $Student(Sno, Sdept, Dname)$ 中，$Sno \to Sdept$，$Sdept \to Dname$，则 $Dname$ 传递依赖于 Sno，不满足 3NF。
解决方法：分解为：

- $Student(Sno, Sdept)$；
- $Dept(Sdept, Dname)$。

4.6 BCNF

BCNF（Boyce-Codd 范式）：在满足 3NF 的基础上，对于每一个非平凡函数依赖 $X \to Y$，X 必须是候选键（超键）。

简单说：**凡是函数依赖的左侧，都必须是候选键。**

BCNF 是比 3NF 更严格的范式，3NF 的关系不一定满足 BCNF。

4.7 范式之间的关系

$$\textbf{BCNF} \subset \textbf{3NF} \subset \textbf{2NF} \subset \textbf{1NF}$$

常见考试方式：给出关系模式和函数依赖集，判断属于哪个范式，并说明原因。

判断方法建议按照：**先判断主键，再看非主属性与主键的依赖关系，最后分析是否有传递依赖或左侧非候选键的情况。**

4.8 函数依赖的公理系统（Armstrong 公理）

考试有时会涉及 Armstrong 公理，共有三条基本规则：

1. **自反律**：若 $Y \subseteq X$，则 $X \to Y$。
2. **增广律**：若 $X \to Y$，则 $XZ \to YZ$。
3. **传递律**：若 $X \to Y$，$Y \to Z$，则 $X \to Z$。

通过这三条规则还可以推导出常用的合并律、分解律等。

4.9 关系数据理论练习

1. 函数依赖 $X \to Y$ 的通俗含义是什么?
2. 完全函数依赖和部分函数依赖的区别是什么？请举例说明。
3. 设关系模式 $R(A, B, C)$，函数依赖集 $F = \{AB \to C, B \to A\}$，$R$ 满足第几范式?
4. 2NF 和 3NF 分别消除了哪类函数依赖问题?
5. 请判断：满足 3NF 的关系模式一定满足 BCNF。（　　）
6. 请判断：在关系模式中，若主键只有一个属性，则该模式一定满足 2NF。（　　）

5 数据库设计

5.1 数据库设计的六个阶段

数据库设计通常分为以下六个阶段：

1. **需求分析**：调查用户需求，收集和分析数据，确定系统边界和目标。

2. **概念结构设计**：将需求抽象成概念模型，通常使用 E-R 模型表达。
3. **逻辑结构设计**：将概念模型转换为特定 DBMS 支持的数据模型，如关系模型。
4. **物理结构设计**：确定数据在存储介质上的组织方式，包括索引、存储路径等。
5. **数据库实施**：根据设计建立数据库，录入数据，测试应用程序。
6. **数据库运行与维护**：长期运行中监控性能，进行调整和维护。

考试中最常考的是前三个阶段，尤其是**概念结构设计**和**逻辑结构设计**。

5.2 E-R 模型基础

概念结构设计的核心工具是 **E-R 模型（实体-联系模型）**。
E-R 图中的三种基本元素：

- **实体**：用矩形框表示，例如" 学生"、" 课程"；
- **属性**：用椭圆形表示，连接到对应实体；
- **联系**：用菱形框表示，连接相关实体，并标注联系类型（1:1、1:n、m:n）。

5.2.1 联系的三种类型

- **一对一（1:1）**：一个实体实例只与另一方的一个实例相关联。例如：一个班级只有一个班长，一个班长只管理一个班级。
- **一对多（1:n）**：一个实体实例与另一方多个实例相关联。例如：一个老师讲授多门课程。
- **多对多（m:n）**：两方实体实例之间互为多关系。例如：一个学生选修多门课程，一门课程有多个学生选修。

5.3 E-R 图转换为关系模式

将 E-R 图转换为关系模式时，基本规则如下：

1. 每个**实体**转换为一个关系模式，实体属性转为关系属性，实体的主标识符转为主键。
2. **1:1 联系**：可将联系的属性合并到任一方关系中。
3. **1:n 联系**：将联系及" 一" 端的主键加入" 多" 端关系中（即在" 多" 端加外键）。
4. **m:n 联系**：联系需单独转换为一个关系模式，包含两端实体的主键（作为联合主键）和联系自身的属性。

例如，学生与课程之间的" 选修"（m:n 联系）需要单独建立选课关系：

$$SC(Sno, Cno, Grade)$$

其中 (Sno, Cno) 为联合主键，Sno 和 Cno 分别是外键。

5.4 数据库设计练习

1. 数据库设计分为哪六个主要阶段?
2. 概念结构设计阶段的主要任务是什么？产出物是什么?
3. E-R 图中矩形、椭圆和菱形分别表示什么?
4. 三种联系类型（1:1、1:n、m:n）在向关系模式转换时分别采用什么策略?
5. 若有实体”部门”（部门号、部门名）和”员工”（员工号、姓名），且一个部门有多个员工，应如何设计关系模式?
6. 请判断：多对多联系在转换为关系模式时，不需要单独建立一个关系。（　　）

6 数据库恢复与并发控制

6.1 事务

6.1.1 事务的定义

事务（Transaction）是用户定义的一个数据库操作序列，这些操作要么全部执行，要么全部不执行，是一个不可分割的工作单位。

6.1.2 事务的 ACID 属性

事务具有四个基本属性，必须全部掌握：

1. **原子性（Atomicity）**：事务中的所有操作要么全部成功，要么全部撤销，不存在部分执行的情况。
2. **一致性（Consistency）**：事务执行前后，数据库必须从一个一致状态转移到另一个一致状态。
3. **隔离性（Isolation）**：一个事务的执行不能被其他并发事务所干扰，各事务之间互相隔离。
4. **持续性（Durability）**：事务一旦提交，其对数据库的改变就是永久性的，即使系统发生故障也不会丢失。

记忆口诀：**原子一致隔离持久**（A-C-I-D）。

6.2 故障类型

数据库中可能遇到的故障主要有三类：

1. **事务故障**：事务执行过程中由于逻辑错误或系统错误导致事务非正常终止。
2. **系统故障**：整个数据库系统崩溃，内存中的数据丢失，但磁盘数据完好。
3. **介质故障**（磁盘故障）：存储设备损坏，磁盘数据丢失，是最严重的一类故障。

6.3 恢复技术

恢复技术的两个核心手段：

- **日志（Log）**：记录每一次数据更新操作的内容（更新前后的值）；
- **备份（Backup）**：周期性地将数据库转储到安全介质。

恢复时的两种基本操作：

- **Undo**（撤销）：将已执行但未提交的事务的操作撤销，还原数据；
- **Redo**（重做）：将已提交但尚未写入磁盘的事务操作重新执行。

6.3.1 检查点技术

检查点技术可以减少恢复时需要扫描的日志范围，提高恢复效率。发生故障后，只需从最近的检查点开始处理，而不是从头扫描整个日志。

6.4 并发控制

6.4.1 并发操作引发的问题

多个事务并发执行时可能出现以下三类问题：

1. **丢失修改**：两个事务同时读取并修改同一数据，其中一个事务的修改被另一个覆盖而丢失。
2. **不可重复读**：一个事务两次读取同一数据，其间另一个事务修改了该数据，导致两次读取结果不一致。
3. **读脏数据**：一个事务读取了另一个事务未提交的修改数据，若后者回滚，则读到了无效数据。

6.4.2 封锁技术

解决并发问题的主要机制是**封锁**：

- **排他锁（X 锁、写锁）**：持锁事务可读可写，其他事务不能再加任何锁。
- **共享锁（S 锁、读锁）**：持锁事务只读，其他事务可以再加 S 锁，但不能加 X 锁。

6.4.3 两段锁协议

两段锁协议（2PL）是保证事务可串行化的常用协议：

- **加锁阶段**：事务可以申请并获取锁，不能释放任何锁；
- **释放锁阶段**：事务可以释放锁，不能再申请新锁。

© 2026 isomoes

6.5 恢复与并发控制练习

1. 事务的 ACID 属性分别是什么？请各用一句话说明。
2. 数据库故障可分为哪三类？其中哪类最严重？
3. Undo 和 Redo 分别在什么情况下使用？
4. 并发事务可能引发哪三类问题？
5. 排他锁和共享锁有什么区别？
6. 请判断：只要事务提交，其对数据库的改变就一定是永久性的，这体现了事务的持续性。（　　）

7 易混点总结

1. **安全性与完整性**：安全性防止**非法用户**访问，完整性防止**合法用户输入不合语义**的数据。
2. **实体完整性与参照完整性**：前者约束**主键不为空**，后者约束**外键合法性**。
3. **2NF 与 3NF**：2NF 消除部分函数依赖，3NF 消除传递函数依赖。
4. **3NF 与 BCNF**：3NF 允许主属性之间存在函数依赖，BCNF 不允许。
5. **概念结构设计与逻辑结构设计**：前者得到 E-R 图，后者得到关系模式。
6. **m:n 联系转换**：必须单独建立一个关系，不能合并到任何一方。
7. **Undo 与 Redo**：未提交事务用 Undo，已提交但未写磁盘的用 Redo。
8. **X 锁与 S 锁**：X 锁排他，S 锁共享，多个事务可同时持有 S 锁但不能同时持有 X 锁。

8 本讲知识框架

建议把本讲整理成下面这条主线：

主题	核心问题	典型内容	高频考点
安全性	谁能访问数据	DAC、GRANT、REVOKE	权限语句、DAC 与 MAC
完整性	数据合不合规	实体、参照、用户定义	三类完整性判断
关系数据理论	模式设计合不合理	函数依赖、1NF～BCNF	范式判断、模式分解
数据库设计	如何系统地建库	六阶段、E-R 图、关系转换	E-R 图绘制与转换
恢复与并发	故障和并发如何处理	ACID、日志、封锁	ACID 属性、并发问题

© 2026 isomoes

9 典型例题

9.1 例题 1：安全性与完整性的区别

题目：数据库安全性和数据库完整性的主要区别是什么?

分析：两者都是保护数据库，但针对的对象不同。

答案：安全性是防止非法用户对数据库的非授权访问；完整性是防止合法用户向数据库中录入不合语义的数据。

9.2 例题 2：判断范式

题目：设关系模式 $R(Sno, Cno, Sname, Grade)$，函数依赖集 $F = \{(Sno, Cno) \to Grade, Sno \to Sname\}$。$R$ 属于哪个范式?

分析：主键为 (Sno, Cno)，$Sname$ 仅依赖于 Sno，存在非主属性对主键的部分函数依赖。

答案：R 满足 1NF，但不满足 2NF。

9.3 例题 3：模式分解

题目：将上题中的 R 分解为满足 2NF 的关系模式。

分析：将部分依赖的属性 $Sname$ 与其完全依赖的属性 Sno 单独成表。

答案：分解为 $Student(Sno, Sname)$ 和 $SC(Sno, Cno, Grade)$。

9.4 例题 4：E-R 图转关系模式

题目：若存在" 学生" 和" 课程" 两个实体，它们之间是多对多的" 选修" 联系，且选修联系含属性" 成绩"，请将其转换为关系模式。

分析：m:n 联系需单独建立关系，包含两端主键及联系属性。

答案：

- $Student(Sno, Sname, \ldots)$
- $Course(Cno, Cname, \ldots)$
- $SC(Sno, Cno, Grade)$，其中 (Sno, Cno) 为主键。

9.5 例题 5：ACID 属性判断

题目：银行转账操作中，从账户 A 扣款并向账户 B 入款，这两个操作必须同时成功或同时撤销，体现了事务的哪个属性?

分析：要么全部执行，要么全部不执行，是原子性的典型描述。

答案：原子性（Atomicity）。

© 2026 isomoes

9.6 例题 6：并发问题判断

题目： 事务 T_1 读取数据 $x = 100$，随即事务 T_2 也读取 $x = 100$，T_1 将 x 改为 200 并提交，T_2 也将 x 改为 120 并提交，最终 $x = 120$ 而非 200。这属于哪类并发问题?

分析： T_1 的修改被 T_2 覆盖而丢失，属于丢失修改。

答案： 丢失修改。

10 课堂练习

10.1 基础练习

1. 数据库安全性控制的主要目标是什么?
2. 三类数据库完整性约束分别是什么？各约束什么?
3. 什么是函数依赖？请用一个生活例子说明。
4. 数据库设计有哪六个阶段？各阶段的主要任务是什么?
5. 事务的四个基本属性分别是什么?

10.2 判断与选择

1. 数据库安全性和完整性都是为了防止非法用户访问数据库。（　　）
2. 主键属性允许取 NULL 值。（　　）
3. 满足 3NF 的关系模式一定满足 BCNF。（　　）
4. E-R 图中的多对多联系转换为关系模式时，需要单独建立一个关系。（　　）
5. 事务提交后，即使系统故障，其对数据库的修改也应该被保留。（　　）
6. 两段锁协议可以解决所有并发控制问题。（　　）

10.3 计算与分析

1. 设关系模式 $R(A, B, C, D)$，函数依赖集 $F = \{A \to B, B \to C, A \to D\}$，请指出 R 的主键，并判断 R 属于哪个范式。
2. 将上题的 R 分解为满足 3NF 的关系模式，并说明分解理由。
3. 描述 Armstrong 公理的三条基本规则，并说明它们的用途。
4. 说明排他锁和共享锁的兼容性规则：当一个事务持有 S 锁时，另一个事务能否再申请 X 锁?
5. 请简述数据库备份与日志在故障恢复中各自的作用。
6. Undo 和 Redo 操作分别针对哪类事务状态?

© 2026 isomoes

10.4 综合小题

题目：某图书馆系统需要管理" 读者" 和" 图书" 两类实体，相关信息如下：

- 读者：读者号、姓名、联系电话；
- 图书：书号、书名、作者；
- 一位读者可以借阅多本图书，一本图书可以被多位读者借阅；借阅关系含属性" 借阅日期"。

请完成下面三个问题：

1. 该系统中，读者与图书之间是什么类型的联系（1:1、1:n 还是 m:n）？
2. 请根据描述设计对应的关系模式（给出每个关系及其属性，标注主键）。
3. 若" 借阅" 关系为 $Borrow(Rno, Bno, BorrowDate)$,其中 (Rno, Bno) 为主键,请说明 $BorrowDate$ 是否完全依赖于主键，并判断该关系满足几范式。

11 课后小测参考答案

1. 确保只有合法的用户以合法的方式访问数据。
2. 实体完整性约束主键非空，参照完整性约束外键合法，用户定义完整性约束具体业务规则。
3. 函数依赖 $X \to Y$ 是指当 X 的值确定后，Y 的值也唯一确定。例如，学号确定后姓名唯一确定。
4. 需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、数据库运行与维护。各阶段依次完成从需求收集、建模、转换到实现和维护的工作。
5. 原子性、一致性、隔离性、持续性。
6. 第 1 题错（完整性针对合法用户的不合规输入）；第 2 题错；第 3 题错；第 4 题对；第 5 题对；第 6 题错（两段锁不能完全避免死锁等问题）。
7. R 的主键为 A，存在传递依赖 $A \to B \to C$，故 R 满足 2NF 但不满足 3NF。
8. 分解为 $R_1(A, B, D)$ 和 $R_2(B, C)$，消除了传递依赖 $A \to B \to C$。
9. Undo 针对已开始但未提交的事务，Redo 针对已提交但未写入磁盘的事务。

综合小题参考:第 1 问为 m:n;第 2 问关系模式为 $Reader(\underline{Rno}, Rname, Tel)$、$Book(\underline{Bno}, Bname, Author)$、$Borrow(\underline{Rno, Bno}, BorrowDate)$；第 3 问 $BorrowDate$ 完全依赖于联合主键 (Rno, Bno)，该关系满足 3NF（非主属性只依赖于主键，无传递依赖）。

11.1 分节练习参考答案

安全性练习参考：

1. DAC 由数据拥有者决定访问权限；MAC 由系统按安全标签强制执行，用户不能干预。
2. 授予 user1 对 Student 表的查询（SELECT）和插入（INSERT）权限。
3. 允许被授权用户将该权限再转授给其他用户。
4. `REVOKE SELECT ON Student FROM user1;`
5. 错。自主存取控制中，权限由数据拥有者或授权者决定。
6. 错。`CASCADE` 会级联回收该用户转授出去的权限。

完整性练习参考：

1. 实体完整性要求主键不为空，因为主键负责唯一标识元组，若为空则无法识别。
2. 参照完整性约束的是外键属性。
3. 外键取值要么等于被参照关系中某个主键值，要么（若允许）为 NULL。
4. 用户定义完整性是针对具体业务的约束，前两类是关系模型本身要求的通用约束。
5. `CHECK` 用于定义属性取值范围，例如 `CHECK (Grade >= 0 AND Grade <= 100)`。
6. 错。若业务允许，外键可以取 NULL 值。

关系数据理论练习参考：

1. X 的值唯一确定后，Y 的值也随之唯一确定。
2. 完全函数依赖：X 的任何真子集都不能单独确定 Y；部分函数依赖：存在 X 的真子集可以确定 Y。例如 $(Sno, Cno) \to Grade$ 是完全依赖，$Sno \to Sname$ 是对 (Sno, Cno) 的部分依赖。
3. AB 为主键；$B \to A$ 说明 B 也可以确定 A，即 A 部分依赖于主键 AB，R 满足 1NF 但不满足 2NF。
4. 2NF 消除部分函数依赖，3NF 消除传递函数依赖。
5. 错。3NF 不一定满足 BCNF。
6. 对。单属性主键不存在部分依赖，因此至少满足 2NF。

数据库设计练习参考：

1. 需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、运行与维护。
2. 主要任务是将需求抽象成概念模型；产出物是 E-R 图。

3. 矩形表示实体，椭圆表示属性，菱形表示联系。

4. 1:1 联系可合并入任一方；1:n 联系在多端加外键；m:n 联系需单独建立关系。

5. $Dept(\underline{Dno}, Dname)$，$Employee(\underline{Eno}, Ename, Dno)$，其中 Dno 为外键。

6. 错。m:n 联系必须单独建立关系，以保存两端主键及联系属性。

恢复与并发控制练习参考：

1. 原子性：全做或全不做；一致性：执行前后数据库保持一致状态；隔离性：事务互不干扰；持续性：提交后永久生效。

2. 事务故障、系统故障、介质故障；介质故障最严重。

3. Undo 用于撤销未提交事务的操作；Redo 用于重做已提交但未写磁盘的操作。

4. 丢失修改、不可重复读、读脏数据。

5. X 锁排他，只有持锁者可读写；S 锁共享，多个事务可同时持有 S 锁，但有 S 锁时不能加 X 锁。

6. 对。这体现了事务的持续性（Durability）。

12 本讲小结

本讲是数据库部分的收尾，涵盖了从**数据保护（安全性、完整性）**到**模式优化（规范化理论）**再到**系统设计（数据库设计）**以及**系统可靠性（恢复与并发控制）**四个维度。

这四个维度共同支撑起一个完整的关系数据库系统：关系模型提供数据表示方式，SQL 提供操作手段，安全性和完整性约束保证数据合法可信，规范化理论保证模式设计合理，数据库设计流程保证从需求到实现的系统性，恢复与并发控制保证系统稳定运行。

至此，数据库原理部分的核心内容已全部覆盖。建议在考前重点回顾：**三类完整性、范式判断方法、E-R 图转换规则、事务 ACID 属性和三类并发问题**。